

SIH26122 — 7-Day Build Plan (v2)

Oil India Limited | Field-Update-to-Gantt Automation

Team of 6 · Zero manual coding · Day 7 = demo-ready

Revision 2 — team roles corrected to actual skill sets, 22 Aug 2026

What Changed in This Revision, and Why

The previous version assigned roles against a wrong skills list. Five things move:

1. **Soumyak is now the primary presenter, not Tarun.** He is the one explicitly identified as a strong public speaker. Tarun becomes secondary presenter and keeps scope authority.
2. **Soumyak is also a real builder now** (some coding + experienced vibe coder), so he is not a dedicated tester any more. He is the second pair of hands on generation alongside Suranjan.
3. **Akshaya's coding ability (Java/C/C++, intermediate) is being used, not wasted on Canva alone.** She becomes the independent verifier of the dependency-recalculation logic — she can read code well enough to catch Arin's errors, which is exactly the check that was missing. She still owns design.
4. **Arin builds, Suranjan drives the tools.** Arin is the strongest coder but weaker with vibe-coding workflows. Pairing them means Arin decides *what correct looks like* and Suranjan gets the AI to produce it fast. Arin should not be fighting Lovable's prompt loop alone.
5. **Wilson replaces Kenneth, and Kenneth's load does not fit him.** Kenneth was carrying schedule authoring, domain taxonomy, test-set creation, evidence policy, deck, and Q&A. Wilson can do decks and structured support work. So: Wilson owns the deck, test execution, and data entry; **Tarun absorbs the domain-knowledge writing** (taxonomy, evidence policy, Q&A) since it is writing and judgement work, not coding, and he has the authority to make those calls anyway.

The honest consequence: this team is stronger on building and weaker on documentation than the previous version assumed. That is a net positive for a hackathon, but it means the domain-credibility layer — the thing that makes your matcher look domain-specific rather than generic — now depends on Tarun protecting time for it. If he lets it slide, you lose your best differentiator and your Q&A prep at the same time.

Corrected Role Map

Person	Actual capability	Role this week
Arin	Strongest coder; less familiar with vibe coding	Technical lead. Owns schema and the dependency-recalculation engine. Reviews all generated code. The correctness authority.
Suranjan	No traditional coding; experienced with Lovable and vibe coding	Primary generation driver. Owns the Lovable app, the matcher prompt, and the AI pipeline. The speed authority.
Soumyak	Some coding; experienced vibe coder; strong speaker	Builder #2 and primary presenter . Takes generation work off Suranjan's queue Days 2–5, then switches to pitch on Day 6.
Akshaya	Canva expert; intermediate Java/C/C++	Design lead and logic verifier. Owns all UI, and independently checks the scheduling maths.
Tarun	PPT; some coding	Team lead, scope controller, domain-knowledge owner (taxonomy, evidence policy, Q&A), secondary presenter.
Wilson	PPT; general assistance	Deck production, synthetic schedule data entry, test-set message generation, test execution, demo asset sourcing.

Two people you cannot afford to lose: Arin (nobody else can judge whether generated scheduling code is correct) and Suranjan (nobody else can drive the tools at speed). Everything else has partial redundancy.

Tool Stack (decided up front — do not re-litigate mid-week)

Layer	Tool	Owner
App generation (frontend + backend)	Lovable.dev (fallback: Bolt.new)	Suranjan, with Soumyak
Database + auth + file storage	Supabase	Arin
LLM calls	Claude API via a Supabase Edge Function	Arin (design) + Suranjan (build)
Gantt rendering	frappe-gantt JS library	Akshaya
Schedule authoring / seed data	Google Sheets → CSV → Supabase	Wilson, spec'd by Arin
Prompt iteration & eval	Claude/ChatGPT web + a Sheets scorecard	Suranjan (prompt) + Wilson (scoring)
Deck & visuals	Canva + Gamma	Wilson + Akshaya, content by Tarun

Hard rule unchanged: every line of code is AI-generated. Humans read, test, and direct. When AI output is wrong, the fix is a better or smaller-scoped prompt — not manual editing. Arin's job is to know *when* it is wrong, which is a different and rarer skill than writing it.

What We're Building (unchanged from v1)

A field engineer types an informal message into a chat box — "*Zone 3 foundation concreting khotom, curing shuru kal se*" — optionally with a photo. The system identifies **which task** in a 40-task schedule it refers to, proposes a **status change**, and puts that proposal in a **review queue**. A site engineer confirms or rejects it. Only on confirmation does the schedule update and **recalculate every downstream dependent task**, redrawing the Gantt chart.

Three parts: an **AI interpreter** (informal language → structured event), a **deterministic scheduling engine** (plain code, no AI), and a **mandatory human gate** between them.

Component	Status in demo
Project schedule	Synthetic but realistic OIL well-site / pipeline EPC schedule
WhatsApp input	Simulated chat UI — Meta approval is a client-side dependency, by design
AI task matching	Real. Live LLM call, measured accuracy on a held-out test set
Photo handling	Real upload + AI description, presented as human-reviewable evidence
Dependency recalculation	Real. Standard forward-pass CPM logic
Human confirmation	Real and mandatory — no status change without it

Differentiation line: OpenSpace, Buildots and Track3D need dedicated 360° capture hardware and a trained operator walking the site. We need a phone and a message someone was sending anyway.

DAY 1 — Foundation: The Schedule Is Real Before Anything Else Is

Goal for the day

A realistic 40-task synthetic Oil India project schedule exists as structured data, sits in a live Supabase database, and renders as a static Gantt chart in a deployed Lovable app. Nothing is intelligent yet — but the thing the AI will modify is real and visible.

What we're building and how

Track A — the schedule dataset. In Google Sheets, a plausible EPC schedule for an OIL well-site development in the Duliajan area: site preparation → civil foundations → mechanical erection → piping → electrical & instrumentation → pre-commissioning. 40 tasks with `task_id`, `wbs_code`,

`task_name`, `zone`, `discipline`, `planned_start`, `planned_finish`, `duration_days`, `predecessor_ids`, `dependency_type` (Finish-to-Start only for now), `percent_complete`, `status`, `responsible_crew`. Dependencies must chain at least 6 levels deep or Day 3's cascade demo has nothing to cascade through. Include 2-3 parallel branches that reconverge — a straight line looks fake to anyone who has used Primavera.

Track B — the app shell. One Lovable project with Supabase attached: `tasks` table matching the schema, plus empty `updates`, `proposals`, and `audit_log` tables. Import the CSV. One read-only page rendering `tasks` through frappe-gantt. Deployed to a live URL by end of day.

Task breakdown by person

- **Arin (lead — schema, before anything else):** Writes the database schema on paper first, so Lovable builds against a decided structure instead of inventing one. Use a separate `dependencies` join table rather than a `predecessors` column — it makes many-to-many predecessors trivial on Day 3 and is the single highest-leverage decision of the day. Owns the Supabase project.
- **Suranjan (lead — generation):** Drives Lovable. Creates the project, wires Supabase, imports the CSV, deploys the read-only Gantt page. Spends 30 minutes at the end walking Arin through how the prompt loop behaves — Arin needs to be able to drive it himself in an emergency.
- **Soumyak (support — generation):** Second Lovable seat. Takes the smaller scaffold tasks (routing, page shells, Supabase client setup) so Suranjan stays on the critical path. Also his day to get familiar with the codebase he will be demoing all week.
- **Wilson (lead — data entry):** Builds the 40-task schedule in Sheets to Arin's schema. Uses AI to draft realistic EPC activity names ("Skid Erection & Alignment," "Hydrotest of Flowline Section 2" — not "Task 14"), then Tarun sanity-checks them. This is a full day's careful work; treat it as a real deliverable, not filler.
- **Akshaya (lead — visual direction):** Sets colour tokens for task status (not started / in progress / complete / at risk / blocked), typography, and the palette. One reference frame in Canva that the Gantt styling matches from Day 3 onward. Then reviews the schedule data for structural errors — circular dependencies, tasks starting before their predecessors finish, orphans. Her coding background makes her the right person for this check.
- **Tarun (lead — scope):** Writes the one-paragraph definition of *done* for the week and the explicit **"not doing" list**: no real WhatsApp API, no Primavera import/export, no multi-project support, no mobile app, no role permissions beyond a single site-engineer view. Every member acknowledges it in writing. This is his single most valuable contribution all week.

Concepts each person should understand today

- **Arin:** Relational modelling of a task graph; why a join table beats an array column for dependencies; Supabase row-level security basics; what he will need to verify in generated code later.
- **Suranjan:** Nothing new — but he should be consciously noting *which* prompt shapes work in Lovable, because he will be teaching Soumyak and Arin from that all week.
- **Soumyak:** The app's structure — where data comes from, which page does what. He is the one who will answer "how does it work" questions on stage, and that answer is far better if he built parts of it.

- **Wilson:** What a WBS is and why codes are hierarchical; Finish-to-Start vs Start-to-Start vs Finish-to-Finish; what float/slack means; roughly how an oil-field surface facility gets built in sequence. Enough to not produce a schedule that an EPC judge finds absurd.
- **Akshaya:** What a Gantt bar must convey at a glance — duration, progress fill, dependency arrows, critical path — so design serves the data. Plus how to spot a cycle in a dependency list.
- **Tarun:** Why over-scoping is the number one cause of hackathon failure, and what a tight 30-second "what it does" sounds like.

End-of-day checkpoint

✓ A public Lovable URL loads a Gantt chart with exactly 40 tasks and visible dependency links, sourced live from Supabase, and Akshaya's dependency audit returns zero errors. If the chart renders but the data is hardcoded rather than queried, **you are behind** — fix it tonight.

DAY 2 — The AI Core: Informal Text → The Right Task

Goal for the day

A messy, code-mixed field message produces structured JSON naming the correct task, the proposed status change, and a confidence score — with **measured** accuracy on a test set you built. This is the day the project either has a real technical claim or doesn't.

What we're building and how

The test set comes first, before the prompt. 60–80 synthetic field messages against the Day 1 schedule, each labelled with its correct `task_id`. Spanning: clean ("Foundation for Zone 3 completed today"), abbreviated ("z3 fdn done"), code-mixed ("zone 3 ka foundation ho gaya, curing kal se"), Assamese-English mixed, deliberately ambiguous ("piping work finished" when three piping tasks exist), partial progress ("about 60% of trenching done"), and negatives (a message about nothing in the schedule, or a delay report). At least 15 should be genuinely hard.

Then the matcher. A Supabase Edge Function `parse-update` that takes message text, injects a compact rendering of the schedule (`task_id`, name, zone, discipline, status — not full rows) into the prompt, and returns strict JSON:

```
{
  "matched_task_id": "C-14",
  "match_confidence": 0.91,
  "alternatives": ["C-15", "C-18"],
  "proposed_status": "complete",
  "proposed_percent": 100,
  "reported_date": "2026-08-22",
  "reasoning": "Message names Zone 3 foundation; C-14 is the only foundation task in Zone 3 and is",
  "needs_clarification": false
}
```

Three deliberate design decisions: (1) **only not-started and in-progress tasks are candidates** — filtering the candidate list is the cheapest accuracy win available; (2) **the model must return alternatives**, because Day 4's review UI turns them into one-click corrections; (3) **low confidence is a valid answer** — a `needs_clarification` flag beats a confident wrong match, and that is the honest-engineering point that plays well with judges.

Task breakdown by person

- **Suranjan (lead — prompt engineering):** Owns the matcher prompt end to end. Runs the test set, tunes, re-runs. Should get through 6-8 iterations today. This is the highest-value work anyone does this week and it is squarely his strength.
- **Arin (lead — plumbing and correctness):** Designs and builds the Edge Function: API key handling (server-side only), schedule-context query, JSON schema validation, graceful degradation when the model returns malformed output, retry logic. His rule to enforce: a bad LLM response must become "needs clarification," never a crash and never a silent wrong write.
- **Tarun (lead — domain taxonomy):** Writes the **Field Update Taxonomy** — the controlled vocabulary of field verbs ("cast," "erected," "hydrotested," "energised," "khotom," "ho gaya," "shuru"), zone and location naming conventions, and how partial progress gets reported verbally on Indian sites. This feeds directly into Suranjan's few-shot examples and is what makes the matcher domain-specific rather than generic. It is also a slide in the deck and an answer in the Q&A. **This is not optional and it is not filler — it is your differentiator.**
- **Wilson (lead — test set construction):** Writes the 60-80 labelled messages against Tarun's taxonomy, using AI to generate variants and then hand-checking every gold label. Then builds the scoring sheet: message, gold label, predicted label, confidence, correct Y/N.
- **Soumyak (support — evaluation):** Runs the scoring passes and — more usefully — produces a **categorised failure list** (wrong zone, wrong discipline, ambiguous phrasing, hallucinated task ID). "78% accurate" is far less useful than "all 9 failures were ambiguous-piping cases," because the second tells Suranjan what to fix.
- **Akshaya (parallel — design):** Designs the chat input component and the proposal card in Canva. How is confidence shown? Not a raw decimal — High/Medium/Low with colour and a non-colour cue. Not implementing yet.

Concepts each person should understand today

- **Suranjan:** Why structured-extraction prompts need an explicit output schema, few-shot examples drawn from real domain vocabulary, and "return null rather than guess" instructions. Why constraining the candidate set beats a cleverer prompt. Temperature near zero and why.
- **Arin:** JSON-mode / structured output enforcement; why you validate model output server-side before it touches the database; why API keys live in the Edge Function and never in the frontend — a judge may well ask.
- **Wilson:** What a labelled evaluation set is, and why building it before the prompt prevents unconsciously tuning to examples you remember. The difference between accuracy and "it worked when I tried it."
- **Soumyak:** How to read an error breakdown and turn it into an actionable statement. Also — since he presents — how to say "we measured 82% top-1 on an 80-message held-out set" with the failure breakdown ready.
- **Tarun:** Enough of the matcher's behaviour to explain it in 45 seconds without saying "it uses AI."
- **Akshaya:** Human-in-the-loop UX — showing enough context for a 5-second decision.

End-of-day checkpoint

✅ The matcher scores $\geq 80\%$ **top-1 accuracy** on the 60+ message test set, the number is written down with a date and prompt version, and Soumyak has a categorised failure list. Under 70% is almost always an unconstrained candidate list or under-specified schedule context — fix that before adding anything else.

DAY 3 — The Engine: Dependency Recalculation and a Live Gantt

Goal for the day

Changing one task's actual finish date — by hand, in the database — correctly pushes every downstream dependent task and redraws the Gantt. Deterministic, explainable, no AI involved.

What we're building and how

A **recalculation function** that runs on every confirmed status change — a forward pass over the dependency graph:

1. Topologically sort tasks by dependency; detect cycles and refuse to run if one exists.
2. `earliest_start = max(actual_finish or projected_finish of all predecessors) + lag`.
3. `projected_finish = earliest_start + remaining_duration`, where remaining duration scales with `percent_complete`.
4. Write `projected_start` / `projected_finish` back, leaving `planned_start` / `planned_finish` untouched — so you can show **baseline vs. current**, the visual that makes slippage obvious.
5. Flag tasks whose projected finish now exceeds their planned finish as **delayed**, and compute the knock-on to project end date.

Explicitly **not** an LLM. Say so in the pitch: "*scheduling maths is deterministic; we use AI only where language is involved.*" That one sentence pre-empts the most common judge objection about AI reliability.

The Gantt then gains: baseline bar (grey, behind) vs. projected bar (coloured), progress fill, dependency arrows, a today-line, and a project-finish readout that visibly moves.

Task breakdown by person

- **Arin (lead):** Owns the recalculation engine end to end. The hardest deterministic piece of the week, on the strongest engineer. He directs AI generation of the topological sort and forward pass, then reads the output line by line. **He should not also be driving Lovable today** — Suranjan or Soumyak handles any app-side wiring so Arin stays on the algorithm.
- **Akshaya (lead — independent verification, critical):** Before Arin's code exists, she hand-calculates 5 test scenarios on paper: "*if C-14 finishes 3 days late, then M-02 starts 3 days late, M-05 starts 3 days late, project end moves 12 Nov → 15 Nov.*" Then she checks the engine's output against her numbers. Her Java/C++ background means she can also read the generated code and spot an off-by-one in the date arithmetic — which is exactly the bug this kind of generated code produces. **This pairing exists so Arin is not the sole verifier of his own work.** It is the most important structural change in this revision.
- **Akshaya (also — visualisation):** Implements the baseline-vs-projected Gantt matching her Day 1 design frame. Dependency arrows, delay highlighting, project-end readout. Heavy day for her; if it is too much, the verification work takes priority and the visual polish slips to Day 5.
- **Suranjan (lead — matcher round 2):** Works Soumyak's Day 2 failure categories into prompt improvements. Adds partial-progress parsing ("about 60% done" → `percent_complete: 60`) since the engine now consumes it.
- **Soumyak (support — app wiring):** Handles Lovable-side work so Arin and Akshaya stay on their specialisms. Also starts internalising the demo narrative — he is presenting this.
- **Tarun (lead — evidence policy + mid-week scope review):** Writes the **Evidence & Confirmation Policy**: who is authorised to confirm a status change, what counts as sufficient evidence per discipline (a photo suffices for civil completion; a hydrotest needs a reported pressure and duration; electrical energisation needs a permit reference), and audit-trail requirements. This turns "we added a Confirm button" into "we designed a governance layer." Then: honest mid-week scope review against this plan, and decide **now** what gets cut.
- **Wilson (support):** Sources or generates 10–12 plausible site photos matched to specific schedule tasks, ready for Day 4. Starts deck skeleton.

Concepts each person should understand today

- **Arin:** Forward-pass scheduling / CPM basics; topological sort and cycle detection; why baseline and projected dates are stored separately; lag and lead.
- **Akshaya:** How to hand-compute a schedule cascade accurately enough to catch a one-day error. Why a Gantt shows baseline behind actual. Why delayed tasks need colour *and* a non-colour indicator — accessibility, and worth 10 seconds in the pitch.
- **Suranjan:** How to convert a failure-category list into a specific prompt change rather than guessing.

- **Tarun:** Audit trails, and why regulated/PSU environments demand attributable changes — who changed what, when, on what evidence.
- **Soumyak:** Enough of the recalculation logic to narrate the cascade moment live and answer a follow-up.

End-of-day checkpoint

✓ Manually setting one mid-chain task to complete-3-days-late cascades correctly through at least 4 downstream tasks, the project end date shifts by exactly the amount Akshaya calculated on paper, and the chart shows baseline vs. projected side by side. Cycle detection tested by deliberately introducing a circular dependency.

DAY 4 — Photo Evidence, the Human Gate, and Deliberate Slack

Goal for the day

The full ingestion path works: a message (with optional photo) creates a **proposal**, the proposal appears in a **review queue**, and confirming it triggers Day 3's recalculation. **The second half of this day is protected buffer.**

What we're building and how

Morning — photo flow. Photo uploads to Supabase Storage. A vision call extracts only what is *observable*: a plain-language description, any readable signage or board text (Indian project sites usually have boards carrying location and activity), and visible discipline cues (rebar, formwork, flanges, cable trays). That output serves two purposes: weak supporting signal to the matcher, and — more importantly — **a caption displayed beside the photo in the review queue**, so the reviewer is not squinting at a thumbnail.

The framing must be enforced in the UI copy. The system never says "verified complete." It says: *"Evidence attached — AI reads this as: formwork removed, concrete surface visible, Zone 3 board legible. Confirm status change?"* If a judge finds a string in your product claiming autonomous verification, the honesty of the whole pitch is in question.

Also morning — the review queue. A page listing pending proposals: the original message, photo + caption, matched task with confidence, top 2-3 alternatives as one-click corrections, the proposed change in plain English, and **Confirm / Correct / Reject**. Confirm writes the status change, fires recalculation, and logs to `audit_log` with user, timestamp, evidence reference, and original message text.

Afternoon — BUFFER. Scheduled slack, placed mid-week deliberately rather than at the end. Realistically Days 2-3 overrun. If everything is genuinely on time, the afternoon goes to the highest-value item on a short list Tarun keeps: matcher accuracy above 90%, a second demo scenario, or a delay-reporting path ("material not received, work stopped") that pushes dates *without* claiming progress — a strong extra beat if it comes free.

Task breakdown by person

- **Arin (lead):** Proposal → confirmation → recalculation pipeline, and the audit log. Transactional integrity is his call: a Confirm must either fully apply or fully fail, and a double-confirm must not double-apply.
- **Suranjan (lead — vision):** Wires photo upload and the vision extraction call. Tunes the prompt to **describe, not conclude** — the literal instruction is "describe what is visible; do not state whether work is complete."
- **Akshaya (lead — review queue UI):** Builds the review queue. This screen is where the product's judgement is visible, so it gets more design attention than any other screen this week. Owns its copy jointly with Tarun.
- **Soumyak (lead — edge-case testing):** Tests the ugly paths: confirming twice, rejecting after confirming, a proposal for an already-complete task, a photo with no message, a message matching nothing. The bug that kills a live demo is nearly always an untested state, not an unbuilt feature — and he is the one who will be standing there when it happens.
- **Wilson (support):** Loads the demo photos, writes the UI microcopy for the confirmation step from Tarun's evidence policy, and drafts the honest-limitations slide.
- **Tarun (lead — buffer control):** Decides at 1pm whether the afternoon is buffer or bonus work. **Nobody else makes this call.** If any Day 1–3 checkpoint is still unmet, the answer is buffer, without discussion.

Concepts each person should understand today

- **Suranjan:** What vision models do reliably (describe, read text, identify objects) versus unreliably (judge completion, assess quality, verify claims). This distinction is the intellectual core of the design.
- **Akshaya:** Human-in-the-loop review UX — enough context for a decision in under 5 seconds, the safe action easy and the risky action deliberate.
- **Arin:** Transactional writes and idempotency.
- **Soumyak:** Edge-case testing as a discipline, plus — as presenter — knowing precisely which states are fragile so he never walks the demo into one.
- **Tarun:** How to say no to good ideas on Day 4.
- **Wilson:** Why "AI-assisted, human-approved" is a stronger claim than "automated," and how to write UI copy that never overstates.

End-of-day checkpoint

✅ End-to-end without touching the database directly: type a message → attach a photo → proposal appears with caption and alternatives → Confirm → task updates → dependents shift → Gantt redraws → audit log records it. If this works, you are on schedule with three days left, which is comfortable.

DAY 5 — The Product Around the Engine

Goal for the day

It stops looking like a prototype. Coherent dashboard, convincing chat input, visible audit history, and a project-health view a manager would actually open.

What we're building and how

The chat input, styled as a recognisable messaging thread — sender name, crew designation, timestamps, bubbles, photo attachment, and system replies inline ("Got it — I think this is C-14 Foundation Concreting, Zone 3. Sent to your site engineer for confirmation."). Two panes side by side in the demo layout: field view left, schedule view right, so the judge sees cause and effect in one glance without tab-switching. **This layout decision is worth more to the demo than any feature built this week.**

The dashboard: project completion %, tasks at risk, projected finish vs. baseline finish with variance in days, pending proposal count, and a recent-activity feed from the audit log.

Task detail view: click any Gantt bar → its history, the messages and photos that changed it, who confirmed each change and when. This is the screen that answers "how do we know this is trustworthy" — having it ready before the question is asked is worth a lot.

Empty, loading and error states, because a demo showing a spinner forever is a failed demo.

Task breakdown by person

- **Akshaya (lead):** Owns everything visual today — dashboard, chat pane, task detail, all states, and behaviour at projector resolution. Her highest-leverage day.
- **Suranjan (support — generation speed):** Pairs with Akshaya, converting her designs into Lovable prompts fast. She decides what it should look like; he makes the AI produce it. This division matters — she should not be fighting the prompt loop and he should not be making design calls.
- **Soumyak (support — generation):** Second seat on the same work, taking the independent screens (task detail, dashboard tiles) so Akshaya and Suranjan can stay on the main demo layout.
- **Arin (lead — hardening):** Steps off features entirely. Loading states, error handling, performance at 40 tasks, and a **one-click demo reset** restoring the database to a known starting state. The reset is not optional — it is what lets the demo run twice, and what saves Day 7 if a rehearsal corrupts the data.
- **Wilson (lead — deck):** Builds the deck: problem framing with OIL context, competitive landscape and differentiation, architecture diagram, honest-limitations slide, evidence/governance policy, business model. Uses **real numbers from Day 2's accuracy testing** — never invented ones.
- **Tarun (lead — script + content):** Supplies the deck's argument and wording to Wilson; writes the demo script beat by beat with timings; decides who speaks which section. **Freezes the feature list at end of day** — nothing new gets built after tonight.

Concepts each person should understand today

- **Akshaya:** Visual hierarchy under demo conditions — what survives a dim projector at 4 metres (contrast, size, one focal point per screen); why side-by-side beats tabbed for demonstrating

causality.

- **Wilson:** How a technical pitch deck is structured — problem, why now, what already exists, what we do differently, how it works, what's real vs. simulated, what's next. And why naming your limitations first defuses the question instead of inviting it.
- **Tarun:** Demo-script craft — open with the pain in one sentence, working system on screen within 90 seconds, one clean surprise (the Gantt cascading), close on differentiation. No feature tours.
- **Arin:** Why a reset-to-known-state function matters more than any remaining feature.
- **Soumyak:** Regression awareness — new UI work routinely breaks working features, and "it worked yesterday" is a claim needing re-verification.

End-of-day checkpoint

✓ Someone from another team, handed the URL with no explanation, understands what the product does within 60 seconds — test this literally and time it. The demo-reset button works and has been used at least twice. Feature list frozen in writing.

DAY 6 — Integration and the First Honest End-to-End Run

Goal for the day

The whole system runs start to finish, three times, without anyone touching the database. Every defect logged and triaged. No new features.

What we're building and how

Morning — demo dataset freeze. Tarun and Wilson fix the exact demo sequence: which 4–5 messages get sent, in what order, with which photos, producing which cascade. It should build: (1) a clean obvious match; (2) an ambiguous message where the system surfaces alternatives or asks for clarification — **deliberately included**, because showing uncertainty handling is more persuasive than three easy wins; (3) a photo-backed completion; (4) a delay report that visibly pushes the project end date. Every message text and photo frozen and stored. No improvising during the pitch.

Midday — three full run-throughs, on the actual demo laptop and, if possible, the actual projector. Reset between each. Everyone present. Log everything that stutters, mismatches, or looks wrong.

Afternoon — triage and fix. Arin and Suranjan work the defect list in severity order. Anything breaking the frozen demo path is P0. Anything cosmetic off the demo path is P2 and probably never gets fixed — say that explicitly so nobody burns Day 7 on it.

Also today — the fallback. Record a full screen-capture video of a successful end-to-end run, and export static screenshots of every key screen into the deck as backup slides. Venue wifi fails, APIs rate-limit, laptops sleep. A team that keeps pitching through a technical failure looks composed; a team that can't looks finished.

Task breakdown by person

- **Arin (lead — defect fixing):** P0 and P1 only. Has authority to refuse any fix that risks the demo path — a Day 6 change touching shared logic is often more dangerous than the bug it fixes.
- **Suranjan (support — second fixer):** Works P1s. Also verifies API quota and rate limits and confirms a spare key exists.
- **Soumyak (lead — test execution, then switch):** Runs the three end-to-end passes and maintains the defect log with severities. **From midday he stops building entirely** and moves to the pitch — he is the primary presenter and cannot be debugging on Day 7 morning. This handover is a scheduled event, not a drift.
- **Wilson (lead — demo dataset + fallback):** Freezes the message and photo set. Records the fallback video. Finishes the deck.
- **Akshaya (support):** Cosmetic fixes on demo-path screens only. Finishes deck visuals and the architecture diagram.
- **Tarun (lead — integration owner):** Runs the day and makes every triage call. First full spoken run-through with Soumyak at day's end.

Concepts each person should understand today

- **Everyone:** The demo path — each person can describe what happens at each step and why, because judges ask whoever is nearest.
- **Soumyak & Tarun:** Presenter/operator split — how a two-person demo stays in sync, and the verbal cue that tells the operator to advance.
- **Arin:** Fix risk assessment on the day before a demo.
- **Wilson:** Why a recorded fallback is a professionalism signal, not an admission of weakness.

End-of-day checkpoint

✅ Three consecutive clean end-to-end runs with no manual database intervention. Defect log with every P0 closed. Fallback video recorded. Deck complete. One full spoken pitch delivered end to end within the time limit.

DAY 7 — Rehearsal, Hardening, and the Story

Goal for the day

Nothing new is built. The team is rehearsed, the hard questions are answered, and the demo has been run enough times that it is boring.

What we're building and how

No feature work. None. The only permitted code changes are fixes to defects that break the frozen demo path, each requiring Tarun's explicit sign-off.

Morning — Q&A preparation. Draft and rehearse these out loud:

- *"How do you know the AI matched the right task?"* → Confidence scoring, alternatives surfaced, mandatory human confirmation. We measured 8x% top-1 on an 80-message test set we built; here is the failure breakdown.
- *"What if the photo doesn't actually show completed work?"* → We never claim it does. The photo is human-reviewable evidence with an AI-generated description. Autonomous visual verification of completion is not a solved problem and we deliberately don't claim it.
- *"OpenSpace and Buildots already do this."* → They require dedicated 360° capture hardware and a trained operator walking the site on a schedule. We ingest a message someone was already sending. Different input cost, different adoption curve, and a fit for Indian PSU/EPC sites where hardware deployment is the blocker.
- *"Is this real Oil India data?"* → No, and we say so first. Synthetic but realistic — here's how we constructed it and why the structure is representative.
- *"Why not use WhatsApp directly?"* → Meta's business verification must be submitted by an authorised representative of the legal entity — that's Oil India, not us. It's a client-side procurement dependency, roughly 3–10 working days once the entity is verified, not an unsolved problem. And our ingestion layer is transport-agnostic: swapping the transport is a webhook, not a rewrite.
- *"What would it cost to run?"* → Almost nothing in message fees. Traffic is inbound-initiated, so replies land inside Meta's 24-hour service window, which has been free and uncapped since Nov 2024. Only proactive nudges outside that window cost anything — Utility templates at roughly ₹0.115 plus GST.
- *"What breaks at scale?"* → LLM cost per message; matching accuracy as task count grows past a few hundred, mitigated by candidate filtering on discipline/zone/status; and review-queue fatigue at high proposal volume, which is why confidence thresholds matter.
- *"What's the business model?"* → Oil India gets free lifetime access under SIH rules. Revenue comes from other EPC and infrastructure firms — per-project subscription, as an ingestion layer on top of whatever PM tool they already run.

Midday — rehearsal block. Minimum five full run-throughs. **Soumyak presents at least three; Tarun presents at least two, complete and alone** — the redundancy only counts if he has actually done it. Time every run. Rehearse the failure branch too: the demo breaks mid-run, switch to the fallback video, keep talking.

Afternoon — logistics, then stop. Charged devices, offline copies of deck and video, mobile hotspot as wifi backup, spare API key, browser tabs pre-opened, notifications off, demo reset run once and left in the starting state. Then stop working. A rested team out-presents a tired one, and Day 7 evening code changes are how demos die.

Task breakdown by person

- **Soumyak (lead — primary presenter):** Owns the pitch. Three full solo run-throughs minimum. Operates nothing during his own presentation — Tarun or Wilson drives the screen.
- **Tarun (lead — rehearsal + secondary presenter):** Runs rehearsals and gives feedback. Delivers two complete solo run-throughs so the pitch survives Soumyak freezing. Owns final Q&A wording and the sign-off on any code change.

- **Wilson (lead — Q&A document + operator):** Compiles the Q&A doc with citations for competitive and market claims. Fact-checks every number in the deck against its source — anything unverifiable comes out. Operates the demo during Soumyak's runs.
- **Arin (on call):** Demo-blocking defects only, with sign-off. Otherwise prepares architecture answers — schema, recalculation logic, why the LLM is not in the scheduling path, where the keys live.
- **Suranjan (on call):** API quota check, spare key, LLM-behaviour questions. Prepares the "how we built this with zero manual coding" answer — it is a legitimately interesting point about the build process if asked.
- **Akshaya (support):** Final deck polish, then joins rehearsals as the hostile-question audience.

Concepts each person should understand today

- **Everyone:** The one-sentence product description and the honest-limitations statement, delivered identically by any team member. Inconsistency between speakers is the fastest way to lose a judge's confidence.
- **Soumyak & Tarun:** Pacing and recovery — what to do when the demo hangs (keep talking, narrate what should be happening, switch to video without apologising three times).
- **Arin & Suranjan:** Defending the architecture in 60 seconds without jargon.
- **Wilson:** Which claims are defensible and which are decoration — and cutting the decoration.

End-of-day checkpoint

✓ Five timed full run-throughs done. Both presenters have delivered the pitch solo end to end. Q&A document covers all eight questions with agreed answers. Fallback video and offline deck on at least two devices. App sitting in its reset starting state. Laptops closed by evening.

Where the Slack Is

Slack	When	How to use it
Primary buffer	Day 4 afternoon	Tarun calls it at 1pm. If any Day 1–3 checkpoint is unmet, it is buffer — not bonus work.
Secondary	Day 6 afternoon	Nominally defect-fixing; absorbs Day 5 overrun if needed.
Not slack	Day 7	Day 7 is rehearsal. Treating it as build time is the most common way hackathon teams lose.

Biggest Risks to This Timeline

1. Day 2's matcher accuracy — highest consequence.

If informal text doesn't map to the right task reliably, there is no product — just a Gantt chart with a chat box next to it. Protect it by having Wilson's test set exist *before* Suranjan's prompt work starts, and by filtering the candidate list hard. If accuracy is still under 70% by end of Day 3, pivot deliberately: cut the demo schedule to 20 clearly-distinguished tasks and lean the pitch on the **human-confirmation workflow** rather than raw matching accuracy. Honest and defensible — but only if decided by Day 3, not Day 6.

2. Day 3's recalculation engine — highest single-point-of-failure.

It lands almost entirely on Arin, and he is the only person who can judge whether generated scheduling code is correct. Mitigations: Arin does nothing else on Day 3; Akshaya's independent paper calculations catch date errors; and if it isn't working by end of Day 4, ship Finish-to-Start-only with fixed durations and no critical-path highlighting — 80% of the visual impact for 30% of the complexity.

3. Arin and Suranjan are both irreplaceable, in different ways.

Arin is the only correctness authority; Suranjan is the only person fluent in the generation tooling. Losing either mid-week is severe. Partial mitigation is cheap and should actually be done: Suranjan spends 30 minutes on Day 1 teaching Arin the Lovable loop, and Arin spends 30 minutes on Day 3 walking Akshaya and Soumyak through the recalculation logic. Neither becomes a replacement, but neither area goes completely dark.

4. Akshaya is over-allocated on Day 3.

She is doing independent verification of the scheduling engine *and* implementing the baseline-vs-projected Gantt. If both cannot fit, **verification wins** and the visual work moves to Day 5. Getting the maths wrong is fatal; an unpolished chart on Day 3 is not.

5. The domain-knowledge layer has a weaker owner than it did.

Taxonomy, evidence policy and Q&A are what make your matcher look domain-specific rather than generic, and they now sit with Tarun alongside his scope-control job. He will be tempted to deprioritise them because they feel like paperwork next to visible building. They are not — they are your differentiator, your credibility with an EPC judge, and your Q&A preparation. Give them real calendar slots on Days 2 and 3.

6. Soumyak is both a builder and the primary presenter.

That is efficient until Day 6, when it becomes a conflict. The scheduled handover — he stops building at midday on Day 6 — must actually happen. A presenter still fixing bugs on Day 7 morning is an unrehearsed presenter.

7. Scope creep between Days 3 and 5.

The most likely *actual* failure mode, and it is social rather than technical. Multi-project support, real WhatsApp integration, mobile apps, Primavera export, role permissions — all will be proposed, all sound reasonable, all are fatal. Tarun's Day 1 "not doing" list and the Day 5 feature freeze are the countermeasures. Enforce both.

8. Demo fragility on Day 7.

Live LLM calls over venue wifi at a specific moment in front of judges. Fallback video, offline deck,

hotspot, spare API key and the reset button are all cheap insurance built on Days 5 and 6. Build them then; you cannot build them on Day 7.

Protect first, if you fall behind: Day 2's matcher and Day 3's recalculation, in that order. Everything else — UI polish, photo captioning, dashboard, second demo scenario — is cuttable. Those two are the product.